Exception Handling and Bank Application with Exception handling

Monday, October 5, 2026 8:09 PM

1. What is an exception?

An exception is an object that represents a failure during program execution. It interrupts the normal flow and transfers control to a matching handler, if one exists.
Exception handling means detecting and responding to failures using try, catch and related language features. Handling a failure does not automatically retry or complete the failed operation.
A critical statement is simply an operation that may fail, such as parsing input, accessing an array element, reading a file or withdrawing money.

	Exception	Error
Meaning	An operation failed; the application may be able to handle it	A serious runtime problem that ordinary applications usually cannot recover from
Example	User enters `"abc"` where a number is expected	JVM runs out of memory
Usual response	Catch the failure and respond appropriately	Investigate the underlying problem rather than catch it as normal application behavior

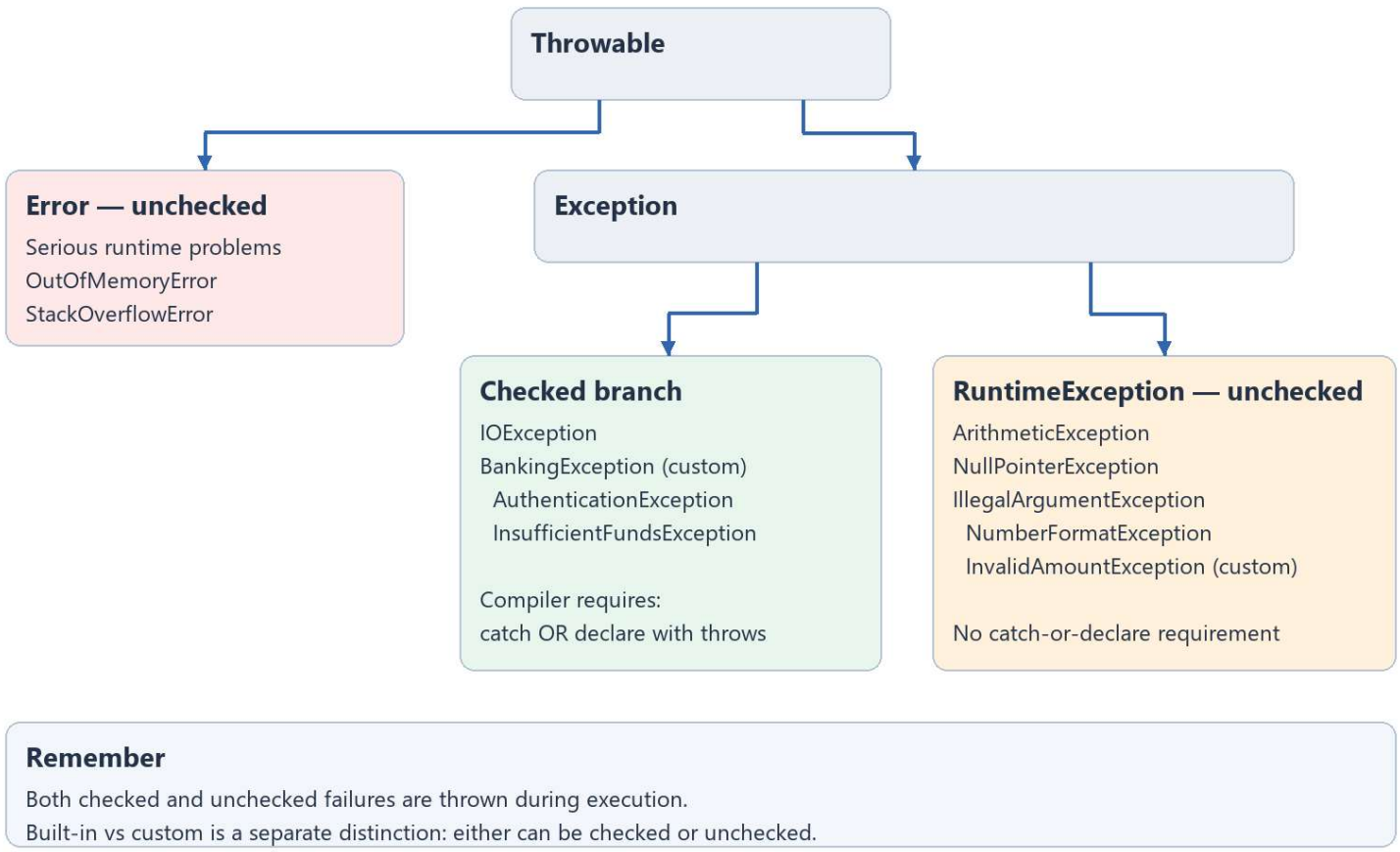
```
int amount = Integer.parseInt("abc"); // NumberFormatException
// This line is not reached if the exception is unhandled:
System.out.println("Amount accepted");
```

2. Compilation errors, logic errors and runtime failures

Compilation error: code does not satisfy Java's syntax or type rules. Example: int amount = "100"; does not compile.
Logic error: code runs but produces the wrong result. Example: adding a delivery charge twice.
Runtime exception: an operation fails while the program runs. Example: Integer.parseInt("abc").
Error is also the name of a Java class for serious runtime problems, such as OutOfMemoryError and StackOverflowError. Ordinary applications generally do not try to recover from these. A compilation error is not an instance of java.lang.Error.

3. Throwable and the exception hierarchy

Throwable is the parent type for objects that can be thrown. Its two main branches are Exception and Error.
Checked exceptions: the compiler requires the caller to catch them or declare them with throws. IOException is an example.
Unchecked exceptions: RuntimeException and its subclasses, and Error and its subclasses. Java does not require catch-or-declare for these.
Both checked and unchecked exceptions occur during execution. “Checked” describes a compiler rule, not when the failure happens.
Built-in versus custom is a separate distinction. A custom exception can be checked or unchecked, depending on its parent class.



4. Common exception examples

ArithmeticException: integer division by zero, such as 10 / zero when zero is 0.
NullPointerException: calling a method through a null reference.
ArrayIndexOutOfBoundsException: accessing an index outside an array's bounds.
NumberFormatException: converting text such as "abc" to an integer.
IllegalArgumentException: a method receives an unacceptable argument, such as a negative duration.
IOException: an input/output operation fails, such as reading a missing or inaccessible file.
An invalid result alone does not automatically become an exception. Business rules must be validated explicitly, with an appropriate exception thrown when a rule is violated.

5. try and catch

try contains the attempted operation. catch receives the first matching exception object and handles the failure.
When a statement throws, the remaining statements in that try block are skipped. Execution does not return to the failed statement after catch.

```
public class ParseDemo {
    public static void main(String[] args) {
        try {
            int amount = Integer.parseInt("abc");
            System.out.println("Parsed: " + amount);
        } catch (NumberFormatException ex) {
            System.out.println("Enter a whole number");
        }
        System.out.println("After the attempt");
    }
}
```

Output: Enter a whole number, then After the attempt. With "25", output is Parsed: 25, then After the attempt.
getMessage() returns the exception's detail message. Printing ex includes its type and message.
printStackTrace() shows call locations for debugging.

6. Catch order and multi-catch

Catch more specific types before their parent types. A parent catch placed first would make the later child catch unreachable, causing a compilation error. This is matching order, not a ranking of exception severity.

```
try {
    Integer.parseInt("abc");
} catch (NumberFormatException ex) {
    System.out.println("Invalid number");
} catch (IllegalArgumentException ex) {
    System.out.println("Invalid argument");
}
```

Multi-catch shares a handler between different types when the response is the same:

```
try {
    int choice = Integer.parseInt("abc");
    int[] options = {10, 20};
    System.out.println(options[choice]);
} catch (NumberFormatException | ArrayIndexOutOfBoundsException ex) {
    System.out.println("Invalid choice");
}
```

Related parent/child types cannot appear together in multi-catch. NumberFormatException | IllegalArgumentException is invalid because the first extends the second.

7. throw and throws

throw raises a particular exception object at a particular line.
throws belongs to a method declaration. It says the method may pass those failures to its caller. It does not catch them.

```
import java.io.IOException;

public class ThrowDemo {
    static void loadStatement(boolean available) throws IOException {
        if (!available) {
            throw new IOException("Statement unavailable");
        }
        System.out.println("Statement loaded");
    }

    public static void main(String[] args) {
        try {
            loadStatement(false);
        } catch (IOException ex) {
            System.out.println(ex.getMessage());
        }
    }
}
```

Output: Statement unavailable. Because IOException is checked, removing both the caller's catch and its throws declaration causes a compilation error.
An unchecked exception may also be named in throws, but Java does not require it.

8. finally and propagation

finally runs as control leaves the associated try/catch, on success or failure, including ordinary return paths. It is useful for cleanup that must happen on both paths.

```
try {
    System.out.println(Integer.parseInt("abc"));
} catch (NumberFormatException ex) {
    System.out.println("Invalid number");
} finally {
    System.out.println("Attempt finished");
}
System.out.println("Program continues");
```

Output: Invalid number, Attempt finished, Program continues.
If no catch matches, finally runs as the failure leaves this method; the failure then travels to the caller. An unhandled exception ends the current thread.
finally is not guaranteed after JVM termination, a process crash or power loss. Avoid returning or throwing from finally because it can replace an earlier result or exception. AutoCloseable resources are usually closed using try-with-resources.

9. Bank application: failures and contracts

This standalone reference implementation demonstrates exception handling in a complete Bank application. It includes checked exceptions and an optional transfer capability for further practice. Use all files together in a separate project with package bank.application. Its constructors use (balance, password, name), and its withdrawal method is withdrawMoney; adapt these signatures when applying the ideas to an existing class project. Each account keeps its balance and password private and validates requests before changing state.
InvalidAmountException: unchecked; zero/negative amounts and balance overflow are rejected.
AuthenticationException: checked; the supplied password is wrong.
InsufficientFundsException: checked; the requested amount exceeds the available balance.
BankingException: checked parent for the two checked Bank failures, allowing a shared

handler.
IllegalArgumentException: unchecked; missing names/passwords, invalid recipients and negative interest years.
These checked/unchecked choices demonstrate Java's rules. Custom business exceptions are not required to be checked.

10. Custom exception classes

Each public class belongs in its own file under bank/application. Calling super(message) stores the message in the parent exception.

InvalidAmountException.java

```
package bank.application;
public class InvalidAmountException extends IllegalArgumentException {
    public InvalidAmountException(String message) { super(message); }
}
```

BankingException.java

```
package bank.application;
public class BankingException extends Exception {
    public BankingException(String message) { super(message); }
}
```

AuthenticationException.java

```
package bank.application;
public class AuthenticationException extends BankingException {
    public AuthenticationException(String message) { super(message); }
}
```

InsufficientFundsException.java

```
package bank.application;
public class InsufficientFundsException extends BankingException {
    public InsufficientFundsException(String message) { super(message)
; }
}
```

11. Interface contracts and account classes

The interfaces declare checked exceptions that callers must handle. An implementation cannot add a broader checked exception than its overridden contract permits. Interfaces can extend multiple interfaces.
BankAccount shares validation and balance updates. SBI and HdfcBank supply their bank names and interest rates. PremiumSBI specializes SBI's behavior.
The interest rates below are classroom sample values.

BankInterface.java

```
package bank.application;
// Banking operations and their checked exception contracts.
// Balance belongs to each account object, not to the interface.
public interface BankInterface {
    int checkBalance(String password) throws AuthenticationException;
    String addMoney(int money);
    String withdrawMoney(int money, String password) throws Authentica
tionException, InsufficientFundsException;
    String changePassword(String oldPassword, String newPassword) thro
ws AuthenticationException;
    double calculateTotalInterest(int years);
}
```

Transferable.java

```
package bank.application;
// Optional capability for transferring money between accounts.
public interface Transferable {
    String transferTo(BankAccount recipient, int money, String passwor
d) throws AuthenticationException, InsufficientFundsException;
}
```

AccountServices.java

```
package bank.application;
public interface AccountServices extends BankInterface, Transferable {
 }
```

SBI.java

```
package bank.application;
public class SBI extends BankAccount {
    public static double rateOfInterest = 7;
    public SBI(int balance, String password, String name) {
        super(balance, password, name);
    }
    @Override public String getBankName() { return "SBI"; }
    @Override protected double getInterestRate() { return rateOfIntere
st; }
```

```
}
```

HdfcBank.java

```
package bank.application;
public class HdfcBank extends BankAccount {
    public static double rateOfInterest = 6;
    public HdfcBank(int balance, String password, String name) {
        super(balance, password, name);
    }
    @Override public String getBankName() { return "HDFC"; }
    @Override protected double getInterestRate() { return rateOfIntere
st; }
}
```

PremiumSBI.java

```
package bank.application;
public final class PremiumSBI extends SBI {
    public PremiumSBI(int balance, String password, String name) {
        super(balance, password, name);
    }
    @Override public String getBankName() { return "Premium SBI"; }
    @Override protected double getInterestRate() {
        return super.getInterestRate() + 0.5;
    }
}
```

12. BankAccount: validate first, then update

Deposits reject invalid amounts before increasing the balance. Withdrawals check authentication, a positive amount and available funds before decreasing it.
Transfers check the sender and recipient before either balance changes. One BankAccount object may access another BankAccount object's private fields because the access occurs inside the same class.
This is a single-thread, in-memory classroom model. Production transfers need database transactions and concurrency controls.

BankAccount.java

```
package bank.application;
import java.util.UUID;

public abstract class BankAccount implements AccountServices {
    private final String name;
    private final String accountNo;
    private String password;
    private int balance;

    protected BankAccount(int balance, String password, String name) {
        if (balance < 0) throw new InvalidAmountException("Opening bal
ance cannot be negative");
        if (name == null || name.isBlank()) throw new IllegalArgumentE
xception("Name required");
        if (password == null || password.isBlank()) throw new IllegalA
rgumentException("Password required");
        this.balance = balance;
        this.password = password;
        this.name = name;
        this.accountNo = UUID.randomUUID().toString();
    }

    public String getName() { return name; }
    public String getAccountNo() { return accountNo; }
    public int checkBalance(String password) throws AuthenticationExce
ption {
        authenticate(password); return balance;
    }
    public String addMoney(int money) {
        if (money <= 0) throw new InvalidAmountException("Amount must
be positive");
        if (money > Integer.MAX_VALUE - balance) throw new InvalidAmou
ntException("Balance limit exceeded");
        balance += money;
        return "Deposited " + money;
    }
    public String withdrawMoney(int money, String password) throws Aut
henticationException, InsufficientFundsException {
        authenticate(password);
        requirePositive(money);
        requireFunds(money);
        balance -= money;
        return getBankName() + ": withdrew " + money;
    }

    public String changePassword(String oldPassword, String newPasswor
```

```
d) throws AuthenticationException {
        authenticate(oldPassword);
        if (newPassword == null || newPassword.isBlank()) throw new Il
legalArgumentException("New password required");
        this.password = newPassword;
        return "Password changed";
    }
    public abstract String getBankName();
    protected abstract double getInterestRate();

    public double calculateTotalInterest(int years) {
        if (years < 0) throw new IllegalArgumentException("Years canno
t be negative");
        // Inherited code uses the actual object's overridden rate.
        return (double) balance * getInterestRate() * years / 100.0;
    }
    public String addMoney(int money, String note) {
        return addMoney(money) + " | Note: " + note;
    }
    public String transferTo(BankAccount recipient, int money, String
password) throws AuthenticationException, InsufficientFundsException {
        // Validate both accounts before changing either balance.
        authenticate(password);
        requirePositive(money);
        requireFunds(money);
        if (recipient == null || recipient == this) throw new IllegalA
rgumentException("Choose another account");
        if (money > Integer.MAX_VALUE - recipient.balance) throw new I
nvalidAmountException("Recipient balance limit exceeded");
        balance -= money;
        recipient.balance += money; // Same class can access private f
ields of another instance.
        return "Transferred " + money + " to " + recipient.getName();
    }
    private void authenticate(String password) throws AuthenticationEx
ception {
        if (!
this.password.equals(password)) throw new AuthenticationException("Wro
ng password");
    }
    private static void requirePositive(int amount) {
        if (amount <= 0) throw new InvalidAmountException("Amount must
 be positive");
    }
    private void requireFunds(int amount) throws InsufficientFundsExce
ption {
        if (amount > balance) throw new InsufficientFundsException("In
sufficient balance");
    }
}
```

13. Handling failures in Main

The account throws the failure; Main decides how to report it. Rejected deposits, withdrawals and transfers leave balances unchanged.

Main declares AuthenticationException for the balance reads that use the sample passwords. A throws declaration propagates a failure; it does not handle it. Each demonstrated failure scenario has a catch block. Interactive applications should catch expected failures where user requests are processed.

Main.java

```
package bank.application;
public class Main {
    public static void main(String[] args) throws AuthenticationExcept
ion {
        // Balance reads propagate AuthenticationException through mai
n.
        // Each failure scenario below has its own catch block.
        BankAccount sender = new SBI(1000, "123", "Asha");
        BankAccount recipient = new HdfcBank(500, "456", "Ravi");
        try { sender.addMoney(-100); }
        catch (InvalidAmountException ex) { System.out.println(ex.getM
essage()); }
        System.out.println("After rejected deposit: " + sender.checkBa
lance("123")); // 1000

        try { sender.withdrawMoney(100, "wrong"); }
        catch (BankingException ex) { System.out.println(ex.getMessage
()); }
        System.out.println("After wrong password: " + sender.checkBala
nce("123")); // 1000

        try { sender.withdrawMoney(1500, "123"); }
```

```
        catch (BankingException ex) { System.out.println(ex.getMessage
()); }
        System.out.println("After insufficient funds: " + sender.check
Balance("123")); // 1000

        try { System.out.println(sender.withdrawMoney(100, "123")); }
        catch (BankingException ex) { System.out.println(ex.getMessage
()); }
        System.out.println("After valid withdrawal: " + sender.checkBa
lance("123")); // 900

        try { sender.transferTo(recipient, 1000, "123"); }
        catch (BankingException ex) { System.out.println(ex.getMessage
()); }
        System.out.println("Rejected transfer: " + sender.checkBalance
("123")
        + " / " + recipient.checkBalance("456")); // 900 / 500

        try { System.out.println(sender.transferTo(recipient, 200, "12
3")); }
        catch (BankingException ex) { System.out.println(ex.getMessage
()); }
        System.out.println("Successful transfer: " + sender.checkBalan
ce("123")
        + " / " + recipient.checkBalance("456")); // 700 / 700

        try { System.out.println(sender.changePassword("123", "789"));
 }
        catch (AuthenticationException ex) { System.out.println(ex.get
Message()); }
        System.out.println("With new password: " + sender.checkBalance
("789")); // 700
        try { sender.checkBalance("123"); }
        catch (AuthenticationException ex) { System.out.println("Old p
assword rejected"); }
        System.out.println("Application continues");
    }
}
```

Expected output

```
Amount must be positive
After rejected deposit: 1000
Wrong password
After wrong password: 1000
Insufficient balance
After insufficient funds: 1000
SBI: withdrew 100
After valid withdrawal: 900
Insufficient balance
Rejected transfer: 900 / 500
Transferred 200 to Ravi
Successful transfer: 700 / 700
Password changed
With new password: 700
Old password rejected
Application continues
```

14. Assignments

Bank practice: implement request validation and custom exceptions in a Bank project. Test zero/negative deposits and withdrawals, wrong passwords, insufficient funds, a valid withdrawal, invalid opening balances and negative interest years. For the optional transfer extension, test a failed and successful transfer and verify both balances. Rejected operations must preserve account state. Submit source files on a feature branch and open a pull request for review.

Vehicles practice: create an abstract Vehicle with registration number and model. Car and Bike extend it and override displayDetails(). A Rentable interface declares rent() and returnVehicle(); rental vehicles implement it. Reject repeated renting or returning with a custom unchecked exception and handle it in Main. This is an original practice exercise.

Listenify practice: design Song and Album, add songs to albums, reject duplicate titles and build a playlist using title or track number. Sketch the relationships between the objects. Collections and playlist navigation are covered in the Listenify resource.

15. Exception handling homework: 15 MCQs

Choose one answer for each question and write a short reason.

1. What is the common parent of Exception and Error? A) Object only B) Throwable C) RuntimeException D) IOException

2. Which is checked? A) IOException B) NullPointerException C) ArithmeticException D) NumberFormatException

3. Checked means: A) failure happens during compilation B) failure is harmless C) compiler requires catch-or-declare D) JVM ignores it

4. Integer.parseInt("abc") throws: A) IOException B) NumberFormatException C) ArithmeticException D) StackOverflowError

5. A statement throws inside try. What happens to later statements in that try? A) skipped B) retried automatically C) always executed D) moved into catch
6. Which catch order is valid for separate catches? A) Exception then NumberFormatException B) IllegalArgumentException then NumberFormatException C) NumberFormatException then IllegalArgumentException D) both A and B
7. Which keyword raises an exception object? A) throws B) throw C) finally D) catch
8. What does throws do? A) catches failures B) retries a method C) declares possible propagated failures D) creates an exception object
9. A custom class extends Exception, but not RuntimeException. It is: A) checked B) unchecked C) an Error D) impossible
10. A custom class extends RuntimeException. Does Java require catch-or-declare? A) yes B) no C) only in constructors D) only in interfaces
11. Which is valid multi-catch? A) Exception | IOException B) IllegalArgumentException | NumberFormatException C) NumberFormatException | ArrayIndexOutOfBoundsException D) RuntimeException | NullPointerException
12. During ordinary try completion, finally: A) runs only on failure B) runs only after a catch C) runs as control leaves D) never runs after return
13. Which may prevent finally from running? A) normal return B) a handled exception C) JVM termination D) successful try
14. A withdrawal fails its insufficient-funds check. The balance should: A) decrease first B) stay unchanged C) become zero D) increase
15. How can Main handle both AuthenticationException and InsufficientFundsException, which extend BankingException? A) catch BankingException B) catch Error C) catch IOException D) use throws as a handler

Book rating practice

Create a Book class containing a title and a rating from 1 to 5.
Requirements:
1. Create a custom checked exception, InvalidRatingException.
2. Implement setRating(int rating). Throw the exception if the rating is outside 1–5.
3. Validate before updating, so an invalid attempt preserves the previous rating.
4. In main, set the rating to 4, then attempt 8. Catch the exception, print its message and show that the rating is still 4.

16. Reading and reference resources

https://www.w3resource.com/java-exercises/oop/index.php

Exception handling — GeeksforGeeks (roadmap reading)
Exception handling — JavaTpoint (roadmap reading)
Oracle: exceptions overview
Oracle: catching exceptions
Oracle: throw
Oracle: throws
Oracle: finally
Oracle: custom exceptions